Listing 9.5 Using a lambda factory function to register a dependency

```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);

builder.Services.AddScoped<IEmailSender, EmailSender>();
builder.Services.AddScoped<NetworkClient>();
builder.Services.AddSingleton<MessageFactory>();
builder.Services.AddScoped(                    ❶
        provider =>                            ❷
            new EmailServerSettings            ❸
            (                                  ❸
                Host: "smtp.server.com",       ❸
                Port: 25                       ❸
            ));                                ❸


WebApplication app = builder.Build();

app.MapGet("/register/{username}", RegisterUser);

app.Run();
```

❶ Because you're providing a function to create the object, you aren't restricted to a singleton.

❷ The lambda is provided an instance of IServiceProvider.

❸ The constructor is called every time an EmailServerSettings object is required instead of only once.

In this example, I changed the lifetime of the created `EmailServerSettings` object to scoped instead of singleton and provided a factory lambda function that returns a new `EmailServerSettings` object. Every time the container requires a new `EmailServerSettings`, it executes the function and uses the new object it returns.

When you use a lambda to register your services, you're given an `IServiceProvider` instance at runtime, called `provider` in listing 9.5. This instance is the public API of the DI container itself, which exposes the `GetService<T>()` and `GetRequiredService<T>()` extension methods you saw in